

# Understanding the TLS Fingerprinting Project

<https://gemini.google.com/app/45ff63bb58f844d2>

*User prompt: Project Description: Build a tool that passively captures TLS ClientHello and ServerHello messages and computes JA3/JA3S fingerprints (with JA4/JA4S as a stretch goal) to identify the client or server application/library generating the traffic. Curate a small reference database and demonstrate distinguishing real clients (browsers, curl, custom scripts) purely from their handshake fingerprint. Tools/Technologies: : libpcap/Scapy (or raw sockets) for capture; fingerprint computation implemented per the JA3/JA4 specifications, eBPF/XDP, Packet/Flow Generators like TRex, KV-stores (Redis, Valkey). Expected Outcome: Demonstrate the working JA3/JA3S fingerprint extractor, validated against published reference JA3 hashes for known clients; A curated database correctly identifying at least 5 distinct clients/tools by fingerprint alone. Demonstrate distinguishing, e.g., curl vs. a browser vs. a custom TLS client on the wire. Understanding on the fingerprinting's role in security monitoring (malware C2 detection, client identification) and its limitations, including fingerprint randomization in modern browsers as an evasion technique. We have to do this project. Can you please explain it to me in detail, what everything means as well.*

Response: At its core, this project involves building a network security monitor that can identify exactly which application (like Chrome, Firefox, curl, or a malicious script) is making a secure connection, simply by looking at the unencrypted initial handshake packets.

Here is a breakdown of the core concepts, the technologies involved, and how they fit together.

## The Core Concepts

- **Passive Capture:** Your tool will listen to network traffic on a network interface without modifying or blocking it, much like a wiretap.
- **TLS ClientHello & ServerHello:** When an application connects to an HTTPS server, it initiates a TLS handshake to agree on encryption parameters. The very first message the client sends is the ClientHello, and the server replies with ServerHello. Because encryption hasn't been established yet, these packets are sent in plain text.
- **JA3/JA3S Fingerprinting:** Different applications use different TLS libraries (e.g., OpenSSL, NSS, BoringSSL) and configure them uniquely. The ClientHello contains a specific list of supported TLS versions, cipher suites, and extensions. JA3 is a standard algorithm that extracts these specific fields, concatenates them, and generates an MD5 hash. Because Chrome's list looks different from curl's list, their JA3 hashes will be entirely different, giving you a unique "fingerprint." JA3S applies the same logic to the ServerHello.
- **JA4/JA4S (Stretch Goal):** A newer, more modular standard that replaces JA3. It uses a structured format that provides more context about the connection (like protocol and SNI) rather than just an opaque MD5 hash.

# The Technology Stack

Given the performance requirements and languages typical for this domain, here is how the suggested tools fit in:

| Technology      | Role in Project                   | Implementation Approach                                                                                                                                                                                                                |
|-----------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| libpcap / Scapy | Packet Capture (Standard)         | Using Python with Scapy is excellent for quickly prototyping the packet sniffer and extracting the TLS fields. For a faster, production-ready C/C++ implementation, libpcap is the industry standard.                                  |
| eBPF / XDP      | Packet Capture (High-Performance) | Extended Berkeley Packet Filter (eBPF) allows you to run sandboxed C code directly inside the Linux kernel to process packets as soon as the network card receives them, bypassing the OS networking stack entirely for extreme speed. |
| Redis / Valkey  | Reference Database                | An in-memory Key-Value store to hold your curated list of known hashes. When your tool generates a JA3 hash, it will instantly query Redis (e.g., GET {ja3_hash}) to return the application name (e.g., "Mozilla Firefox 115").        |
| TRex            | Traffic Generation                | An open-source traffic generator. You can use it to simulate heavy network loads or replay saved packet captures (PCAPs) containing various ClientHello messages to test if your tool can keep up and correctly identify them.         |

# Security Context & Evasion

In the real world, this technique is heavily used by security operations centers (SOCs) to detect malware. Even if malware uses HTTPS to hide its command-and-control (C2) traffic, its custom Python or C script will likely have a different JA3 fingerprint than standard corporate web browsers, making it stick out.

However, modern browsers deploy evasion techniques like **TLS GREASE** (Generate Random Extensions And Sustain Extensibility). They intentionally inject randomized, fake extensions into the ClientHello to prevent middleboxes from rigidly hardcoding expected values. Your JA3 implementation must properly ignore these randomized GREASE values to maintain a consistent fingerprint.

User prompt: *Hi, I have been busy with other things, so please go over this code they have written, and please explain everything that has been implemented. I wanted to get started with GUI, so tell me what will be the best way to do that too.*

Response: Your project implements a network security monitor that identifies client and server applications by analyzing unencrypted TLS handshake parameters (JA3/JA3S/JA4). The codebase provides implementations in both Python and C++ to passively capture network traffic, extract fingerprint metadata, and match it against a database of known clients like browsers or curl.

## Core Codebase Architecture

- **Packet Capture & TCP Reassembly:** The code utilizes dpkt in Python (src/capture.py) and libpcap in C++ (src/capture.cpp) to ingest .pcap files or monitor live traffic. It reconstructs complete data streams by managing out-of-order TCP segments and partial overlaps.
- **TLS Protocol Parsing:** The src/parser.py and src/parser.cpp modules dig into the reassembled streams to locate ClientHello and ServerHello messages. They extract key fields like the TLS version, cipher suites, supported elliptic curves, and extensions.
- **Fingerprint Generation:** The src/ja3 and src/ja4 modules filter out randomized "GREASE" values—fake extensions used by modern browsers to evade fingerprinting—before formatting the remaining fields into standardized strings and computing MD5 or SHA256 hashes.
- **Reference Database:** The src/db.py module manages a JSON-backed key-value store (fingerprints.json) to instantly map these generated hashes to known client applications.

## Approaches for Adding a GUI

To wrap this capture engine in a graphical interface, you have a few optimal paths depending on how you want to handle data ingestion and UI state:

| Framework | Language | Architecture Fit                  | Best For                     |
|-----------|----------|-----------------------------------|------------------------------|
| Streamlit | Python   | Acts as a web dashboard over your | Rapidly building data-heavy, |

| Framework       | Language | Architecture Fit                                                   | Best For                                                                                               |
|-----------------|----------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
|                 |          | capture.py data ingestion scripts.                                 | analytical dashboards with minimal frontend routing.                                                   |
| PyQt6 / PySide6 | Python   | Native desktop application binding directly to your Python engine. | High-performance, traditional desktop monitoring applications (similar to Wireshark).                  |
| Qt / ImGui      | C++      | Direct integration with your tlsfp_engine via CMake.               | Maximum performance live-capture tools requiring custom rendering or translucent window modifications. |

When wiring up the interface events to your backend parsers, applying camelCase naming conventions for your UI elements (e.g., packetDataTableView, hashMatchLabel) will keep the new frontend layer clean and consistent with your standard formatting habits.

Which specific visualization features (like live packet scrolling, pie charts of matched clients, or raw hex views) do you want to prioritize in the interface?

---

*User prompt: well right now I want to know everything that has been done. I dont have the slightest clue what has been done and what it does and what it means. Teach me everything that I need to know about this project.*

**Response:** It is completely normal to feel overwhelmed by a dual-language networking codebase. This project operates as a passive network monitor that identifies applications (like Chrome, Firefox, or custom scripts) by analyzing the unencrypted parameters they use when initiating a secure HTTPS connection.

## The Core Mechanism

Whenever an application connects to a secure server, it sends a ClientHello message outlining its cryptographic capabilities. Since different applications use different TLS libraries, this list acts as a unique fingerprint.

- **Packet Capture:** The tool reads raw network traffic (either live or from a .pcap file).
- **TCP Reassembly:** Network packets often arrive out of order or duplicated. The codebase reconstructs the pristine data stream.
- **TLS Parsing:** It isolates the ClientHello (client) or ServerHello (server) messages and extracts specific

cryptographic fields.

- **Hashing:** It formats these fields into a string and hashes them (JA3 uses MD5; JA4 uses SHA256) to create a compact identifier.
- **Database Lookup:** The hash is checked against a JSON database to identify the software.

## Python Implementation (code/python/)

This is the primary analytical engine, built with a focus on robust parsing and testing.

| File       | Core Responsibility                                                                                                                                                                            |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| main.py    | The command-line interface. Parses user arguments to analyze traffic offline (pcap) or live (live).                                                                                            |
| capture.py | Uses dpkt to parse network layers. Crucially, it includes TCPReassembler, which buffers out-of-order packets and trims overlapping retransmissions to reconstruct the TLS handshake perfectly. |
| parser.py  | Decodes the Type-Length-Value (TLV) byte structures of the TLS protocol to extract versions, cipher suites, and extensions.                                                                    |
| ja3.py     | Formats the parsed fields and computes the MD5 hashes. It specifically filters out "GREASE" values—randomized data browsers inject to evade fingerprinting.                                    |

## C++ Implementation (code/cpp/)

This is the high-performance variant of the tool, executing the same concepts but utilizing libpcap for direct packet capture and OpenSSL for hardware-accelerated hashing.

- **capture.cpp:** Uses an atomic signal handler for safe termination and tracks active network flows using a hash map (std::unordered\_map).
- **ja4.cpp:** Successfully implements the stretch-goal JA4 standard, creating a structured, 36-character fingerprint (e.g., t13d1516h2\_...) rather than a simple MD5 hash.

## Reference Data (code/pcaps/ & code/db/)

The pcaps/ directory contains captured traffic from various clients (e.g., chrome.pcap, curl.pcap) used to validate the engine. The db/ folder contains scripts to compile known JA3/JA4 hashes into the

fingerprints.json reference dictionary.

Which specific component—the TCP reassembly logic, the TLS byte parsing, or the C++ engine—would you like to explore first?

User prompt: okay, I would like to start building a native desktop app for this application. What should I use to do that?

Response: For this project, **PyQt6 (or PySide6)** is the strongest choice if you want to immediately wire up your existing Python parser and reassembler, while **Qt6 with C++** is the route to take if you want zero-copy capture performance linked directly to your CMakeLists.txt and libpcap.

### Framework Comparison

| Framework        | Backend Integration                        | High-Frequency Packet Handling                                | Development Speed | Best For                                                              |
|------------------|--------------------------------------------|---------------------------------------------------------------|-------------------|-----------------------------------------------------------------------|
| PyQt6 / PySide6  | Native Python (src/capture.py, src/ja3.py) | Solid (handles thousands of handshakes via QTableView models) | Fast              | Quick prototyping, rich native widgets, zero C++ compilation overhead |
| Qt6 (C++)        | Native C++ (tlsfp_engine, libpcap)         | Maximum throughput (direct memory buffers, zero IPC)          | Moderate          | Production-grade network monitors, minimal memory footprint           |
| Tauri / Electron | Requires IPC/Subprocess piping to CLI      | High memory overhead; struggles with rapid DOM updates        | Slow to wire      | Web-based styling and CSS dashboards                                  |

### Recommended Architecture: The 3-Pane Monitor

To replicate a clean network analyzer layout (similar to Wireshark or Charles Proxy), structure your UI around three main visual areas:

1. **Top Control Bar:** File picker for .pcap files, interface selector dropdown for live sniffing, a filter input

box, and Start/Stop buttons.

2. **Handshake Table (QTableView):** Displays real-time flow rows (Timestamp, Src IP:Port, Dst IP:Port, SNI, JA3 Hash, Matched Client).
3. **Inspection Pane (QTextEdit or QTreeWidget):** Shows the selected flow's breakdown (raw TLS version, cipher list, curve IDs, full JA3 raw string, and JA4 identifiers).

## Core Concurrency Pattern (Worker Thread)

Network sniffing and packet reassembly are blocking operations. You must run the packet ingestion loop inside a background worker thread (QThread) and communicate with the UI via signals to prevent the interface from freezing.

```
from PyQt6.QtCore import QThread, pyqtSignal, QObject
from src.capture import read_pcap, CaptureStats
from src.ja3 import compute_ja3_hash, compute_ja3_string
from src.db import FingerprintDB

class CaptureWorker(QThread):
    packetParsed = pyqtSignal(dict)
    captureFinished = pyqtSignal(str)

    def __init__(self, pcapPath: str, dbPath: str = "fingerprints.json"):
        super().__init__()
        self.pcapPath = pcapPath
        self.fingerprintDb = FingerprintDB(dbPath)
        self.captureStats = CaptureStats()

    def run(self):
        for result in read_pcap(self.pcapPath, self.captureStats):
            if result.client_hello:
                clientHello = result.client_hello
                ja3Hash = compute_ja3_hash(clientHello)
                matchedApp = self.fingerprintDb.lookup(ja3Hash) or "Unknown"

                packetData = {
                    "source": f"{result.src_ip}:{result.src_port}",
                    "destination": f"{result.dst_ip}:{result.dst_port}",
                    "sni": clientHello.server_name or "N/A",
                    "ja3Hash": ja3Hash,
                    "ja3Raw": compute_ja3_string(clientHello),
                    "matchedClient": matchedApp,
                }
                self.packetParsed.emit(packetData)

        self.captureFinished.emit(self.captureStats.summary())
```

Would you like to build the PyQt6 desktop layout first, or focus on integrating the capture worker with live interface capture?

---

*User prompt: If we build the PyQt6 thing first, then can we wire it to both the python pipeline and the C++ pipeline? Because they are currently working on the C++ one. (It mostly works). Can we build one unified dashboard, and then hook up either, whichever one we want? and if we can, I want to start work with PyQt6. Tell me how to set it up, and make the interface.*

Response: A unified dashboard can easily swap between both pipelines by mapping the Python engine to a background QThread and launching the compiled C++ executable via QProcess. Since you already tweak Windows with custom themes and translucent mods, PyQt6 provides the exact low-level widget control needed to make this network monitor integrate cleanly into your customized environment.

## Setup Requirements

- Install the library via pip install PyQt6.
- Ensure your C++ binary (tlsfp\_engine) is compiled and accessible in the project directory.

## Unified Dashboard Implementation

```
import sys
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QHBoxLayout, QPushButton, QTableView, QTextEdit,
                             QComboBox, QFileDialog)
from PyQt6.QtCore import QProcess

class TlsMonitorWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(1000, 600)

        mainWidget = QWidget()
        self.setCentralWidget(mainWidget)
        mainLayout = QVBoxLayout(mainWidget)

        controlLayout = QHBoxLayout()
        self.backendSelect = QComboBox()
        self.backendSelect.addItem("Python Engine", "C++ Engine (libpcap)")

        self.loadPcapButton = QPushButton("Load PCAP")
        self.loadPcapButton.clicked.connect(self.selectPcapFile)

        controlLayout.addWidget(self.backendSelect)
        controlLayout.addWidget(self.loadPcapButton)
        mainLayout.addLayout(controlLayout)

        self.packetDataTableView = QTableView()
        mainLayout.addWidget(self.packetDataTableView)

        self.hashMatchLabel = QTextEdit()
        self.hashMatchLabel.setReadOnly(True)
        mainLayout.addWidget(self.hashMatchLabel)
```

```

self.cppProcess = QProcess()
self.cppProcess.readyReadStandardOutput.connect(self.handleCppOutput)

def selectPcapFile(self):
    fileName, _ = QFileDialog.getOpenFileName(self, "Select PCAP", "",
"PCAP Files (*.pcap *.pcapng)")
    if fileName:
        self.runEngine(fileName)

def runEngine(self, filePath):
    self.hashMatchLabel.clear()
    if self.backendSelect.currentText() == "C++ Engine (libpcap)":
        self.cppProcess.start("./tlsfp_engine", ["-r", filePath])
    else:
        self.hashMatchLabel.append("Python QThread execution goes here.")

def handleCppOutput(self):
    outputBytes = self.cppProcess.readAllStandardOutput().data()
    outputText = outputBytes.decode("utf-8", errors="ignore")
    self.hashMatchLabel.append(outputText)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    monitorWindow = TlsMonitorWindow()
    monitorWindow.show()
    sys.exit(app.exec())

```

## Wiring Breakdown

- **The C++ Pipeline:** The QProcess class executes your compiled tlsfp\_engine binary asynchronously. Every time your C++ code prints a matched fingerprint to std::cout, the readyReadStandardOutput signal fires, allowing the Python GUI to capture and stream the text into the inspection pane without freezing.
- **The Python Pipeline:** To activate the Python route, you will instantiate a QThread subclass that imports your read\_pcap function. It processes the traffic in the background and emits a pyqtSignal containing a dictionary of the extracted ClientHello fields.
- **The Toggle Mechanism:** The backendSelect dropdown routes the selected file path to either the external C++ process or the internal Python thread, letting you test the parsing accuracy of both languages side-by-side.

What specific C++ standard output data would you like to parse and format into the columns of the packetDataTableView?

---

*User prompt: I dont understand. Lets start again. I have made a venv, where I have installed PyQt6. Now, lets make a new folder GUI in the code folder. There, we can put our GUI code. Now, how do you want me to compile the previous code? What are we doing?*

Response: We are building a control center (your PyQt6 app) that acts as the face of your network monitor. Under the hood, this GUI needs a heavy-lifting engine to actually read network traffic and extract the TLS fingerprints.

You already have two engines written in your code folder: a Python engine and a C++ engine. Right now, we will compile the C++ engine so it becomes a standalone executable. Then, our PyQt6 app will launch that executable in the background, hand it a .pcap file, and read its output to display on the screen.

**Step 1: Compiling the C++ Engine** Since the C++ code relies on libpcap and OpenSSL, it must be compiled first. Assuming you are operating in a Linux environment (like WSL on your Windows setup) based on the apt install notes in your CMakeLists.txt, open your terminal and run:

1. Navigate to the C++ source: `cd code/cpp`
2. Create a build directory: `mkdir build && cd build`
3. Configure the project: `cmake ..`
4. Compile the engine: `make`

This generates an executable named `tlsfp_engine` inside the build folder.

**Step 2: The Initial GUI Skeleton** Create a file named `main.py` inside your new `code/GUI/` folder. We will wire up a basic layout using camelCase naming for all UI elements to keep the frontend clean and consistent.

```
import sys
import os
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QHBoxLayout, QPushButton, QTextEdit, QFileDialog)
from PyQt6.QtCore import QProcess

class TlsFingerprintGui(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(900, 600)

        # Main layout container
        centralWidget = QWidget()
        self.setCentralWidget(centralWidget)
        mainLayout = QVBoxLayout(centralWidget)

        # Top controls
        controlLayout = QHBoxLayout()
        self.loadPcapButton = QPushButton("Load PCAP File")
        self.loadPcapButton.clicked.connect(self.selectPcapFile)
        controlLayout.addWidget(self.loadPcapButton)

        mainLayout.addLayout(controlLayout)

        # Output display
        self.outputTextEdit = QTextEdit()
        self.outputTextEdit.setReadOnly(True)
        self.outputTextEdit.setStyleSheet("background-color: #1e1e1e; color:
```

```

#00ff00; font-family: monospace;")
    mainLayout.addWidget(self.outputTextEdit)

    # Background process for the C++ engine
    self.engineProcess = QProcess()

self.engineProcess.readyReadStandardOutput.connect(self.readEngineOutput)

self.engineProcess.readyReadStandardError.connect(self.readEngineErrors)

    def selectPcapFile(self):
        filePath, _ = QFileDialog.getOpenFileName(self, "Select PCAP",
        "../pcaps", "PCAP Files (*.pcap *.pcapng)")
        if filePath:
            self.outputTextEdit.append(f"[*] Loading {filePath}...\n")
            self.runCppEngine(filePath)

    def runCppEngine(self, pcapFilePath):
        # Path to the executable compiled in Step 1
        executablePath = "../cpp/build/tlsfp_engine"

        if not os.path.exists(executablePath):
            self.outputTextEdit.append("[-] Error: tlsfp_engine not found. Did
you compile the C++ code?\n")
            return

        # Launch the engine with the selected file
        self.engineProcess.start(executablePath, ["-r", pcapFilePath])

    def readEngineOutput(self):
        outputData =
self.engineProcess.readAllStandardOutput().data().decode("utf-8")
        self.outputTextEdit.append(outputData)

    def readEngineErrors(self):
        errorData =
self.engineProcess.readAllStandardError().data().decode("utf-8")
        self.outputTextEdit.append(errorData)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    mainWindow = TlsFingerprintGui()
    mainWindow.show()
    sys.exit(app.exec())

```

**How it fits together** When you click "Load PCAP File", the PyQt6 app locates a network capture file and feeds it to the `tlsfp_engine` binary. As the C++ engine rapidly chews through the network packets and prints out JA3/JA4 hashes, the GUI catches that output in real-time (`readEngineOutput`) and streams it directly into the text box.

Try running this `main.py` and loading one of the `.pcap` files from your `code/pcaps/` folder. Does the data

successfully populate the screen?

---

*User prompt: stop giving me code for now and help me compile. I am on Windows rn. How should I do this.*

Response: Because the project's CMakeLists.txt explicitly relies on libpcap and OpenSSL, the absolute simplest way to compile this on a Windows machine is by using WSL (Windows Subsystem for Linux). Compiling it natively in Windows Command Prompt requires downloading the Npcap SDK, setting up a package manager like vcpkg for OpenSSL, and rewriting parts of the CMake file.

## 1Open WSL

Terminal

Launch your WSL terminal (e.g., Ubuntu) and navigate to the code/cpp directory in your project folder.

## 2Install Dependencies

Prerequisites

Run `sudo apt update && sudo apt install cmake g++ libpcap-dev libssl-dev` to get the required C++ compilers and libraries.

## 3Create Build Directory

Setup

Run `mkdir build && cd build` to create an isolated folder for your compiled files.

## 4Configure and Compile

Build

Run `cmake ..` followed by `make` to generate the `tlsfp_engine` executable.

If you run your PyQt6 Python GUI from inside WSL (or using VS Code's WSL extension), it will be able to launch this Linux executable perfectly while still rendering the GUI window on your Windows desktop.

---

*User prompt: bro how do I navigate to my directory, ubuntu opens up here dawg*

Response: WSL mounts your Windows file system under the /mnt directory.

To access the project folder where your C++ code lives, you need to use the cd (change directory) command to navigate into your Windows C: drive.

- **If your project is on your Windows Desktop:**

```
cd /mnt/c/Users/<YourWindowsUsername>/Desktop/<YourProjectFolder>/code/cpp
```

- **If your project is somewhere else on your C drive:**

```
cd /mnt/c/<path-to-your-folder>/code/cpp
```

Replace <YourWindowsUsername> with your actual Windows account name. You can use the ls command at any point to verify you are in the right place. Once you are in the code/cpp directory, you can proceed with the mkdir build and cmake .. commands.

---

*User prompt: built. Now what*

Response: Since your GUI is running natively in Windows via your Python virtual environment, but your C++ engine was just compiled as a Linux binary inside WSL, we need to bridge the two operating systems. Windows cannot execute a Linux binary directly, but PyQt6's QProcess can invoke it seamlessly using wsl.exe.

**Update the Execution Logic** Open your code/GUI/main.py file and modify the runCppEngine method. Modern WSL automatically handles converting the Windows file path generated by the file dialog into a Linux /mnt/c/... path.

```
def runCppEngine(self, pcapFilePath):
    executablePath = "../cpp/build/tlsfp_engine"

    self.outputTextEdit.append(f"[*] Dispatching to WSL: {executablePath}\n")

    # Prepend wsl.exe to execute the Linux binary from Windows
    self.engineProcess.start("wsl.exe", [executablePath, "-r", pcapFilePath])
```

## Launch the Interface

- Open your standard Windows Command Prompt or PowerShell (not your WSL Ubuntu terminal).
- Verify your Python virtual environment containing PyQt6 is activated.
- Navigate into the code/GUI directory.
- Run python main.py.

Once the window appears on your desktop, click the load button and select cloudflare\_run1.pcap from

your pcaps folder.

Did the TLS fingerprint data successfully stream from the Linux engine into the dark-themed inspection pane, or did it throw a file pathing error?

User prompt: *idk if I like this strategy of having to be dependent on wsl. Because this makes it work in a very specific configuration. Can we do any better*

Response: The reason WSL was forced is because the C++ implementation relies on POSIX-specific Linux headers (<netinet/ip.h>, <arpa/inet.h>, <unistd.h>, and sigaction), which do not exist natively on Windows.

Here is how you can eliminate the WSL dependency completely, ranked from simplest to most robust.

## 1. Drive the GUI with the Pure Python Engine (Recommended)

Your Python backend (code/python/src/) is already 100% implemented, tested, and cross-platform. It uses standard libraries (dpkt, socket, hashlib, json):

- **Zero External Dependencies:** Runs directly inside your current Windows virtual environment without compiling binaries or bridging operating systems.
- **In-Process Speed:** Instead of invoking an external process and parsing raw text output, PyQt6 calls your Python functions directly in a background QThread and receives structured Python objects.
- **Full Portability:** The exact same code runs identically on Windows, macOS, and Linux.

## 2. Port the C++ Engine to Native Windows (.exe)

To compile the C++ engine directly on Windows without WSL, the POSIX dependencies must be replaced with Windows equivalents:

| POSIX Linux Implementation (Current)           | Native Windows Equivalent                                                     |
|------------------------------------------------|-------------------------------------------------------------------------------|
| <netinet/ip.h>, <netinet/tcp.h>, <arpa/inet.h> | <winsock2.h>, <ws2tcpip.h>, plus manual struct definitions for IP/TCP headers |
| libpcap-dev                                    | <b>Npcap SDK</b> (wpcap.lib, Packet.lib)                                      |
| OpenSSL::Crypto via apt                        | OpenSSL installed via vcpkg or pre-built Windows binaries                     |
| getopt, sigaction                              | Windows console control handlers (SetConsoleCtrlHandler)                      |

Once ported, CMake compiles a native `tlsfp_engine.exe` using Visual Studio / MSVC, which your Windows GUI can launch directly without `wsl.exe`.

### 3. Comparison of Approaches

| Approach                | Setup Overhead                     | Windows Native? | Performance             | Best Used For                                    |
|-------------------------|------------------------------------|-----------------|-------------------------|--------------------------------------------------|
| Python Backend in GUI   | None (Ready now)                   | Yes             | High (plenty for PCAPs) | Immediate UI development, cross-platform testing |
| Native C++ Port (Npcap) | Moderate (needs SDKs & code edits) | Yes             | Maximum                 | High-throughput 10Gbps live traffic analysis     |
| WSL Bridge (wsl.exe)    | Low (already built)                | No              | Moderate (IPC overhead) | Quick verification of Linux C++ code             |

### The Cleanest Path Forward

Hook the PyQt6 GUI directly into your Python engine (`src/capture.py` and `src/ja3.py`). This keeps your entire workflow inside your native Windows virtual environment, removes external bridges, and gives you a fully functional application immediately. When your team finishes or ports the C++ engine later, you can plug the native `.exe` in as an optional secondary backend.

Would you like to wire up the PyQt6 GUI directly to the native Python capture engine?

---

*User prompt: let's start again with the PyQtGUI thing with the python engine for now. How should I do that*

Response: Let's drop the WSL complexity and build a fast, native Windows dashboard entirely on top of your existing Python parsers.

**1. Install Dependencies** Ensure your virtual environment is active in your standard Windows terminal, then install PyQt6: `pip install PyQt6`

**2. The Application Code** Create a new file named `gui.py` inside your `code/python/` directory. Placing it here ensures it can easily import your existing `src` modules without any complex pathing issues.

This implementation uses a `QThread` so the interface remains perfectly responsive while your engine processes large `.pcap` files in the background.

```

import sys
from pathlib import Path
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QHBoxLayout, QPushButton, QTableWidgetItem,
                             QFileDialog, QHeaderView)
from PyQt6.QtCore import QThread, pyqtSignal

# Import your existing Python engine
from src.capture import read_pcap, CaptureStats
from src.ja3 import compute_ja3_hash
from src.db import FingerprintDB

class PcapWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    captureFinished = pyqtSignal(str)

    def __init__(self, pcapPath):
        super().__init__()
        self.pcapPath = pcapPath
        self.fingerprintDb = FingerprintDB("fingerprints.json")
        self.captureStats = CaptureStats()

    def run(self):
        for result in read_pcap(self.pcapPath, self.captureStats):
            if result.client_hello:
                clientHello = result.client_hello
                ja3Hash = compute_ja3_hash(clientHello)
                match = self.fingerprintDb.lookup(ja3Hash) or "Unknown"

                packetData = {
                    "source": f"{result.src_ip}:{result.src_port}",
                    "destination": f"{result.dst_ip}:{result.dst_port}",
                    "sni": clientHello.server_name or "N/A",
                    "ja3Hash": ja3Hash,
                    "matchedClient": match
                }
                self.rowExtracted.emit(packetData)

        self.captureFinished.emit(self.captureStats.summary())

class TlsMonitorGui(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(1000, 600)

        mainWidget = QWidget()
        self.setCentralWidget(mainWidget)
        mainLayout = QVBoxLayout(mainWidget)

        # Top Controls

```

```

        controlLayout = QHBoxLayout()
        self.loadButton = QPushButton("Load PCAP File")
        self.loadButton.clicked.connect(self.selectPcapFile)
        controlLayout.addWidget(self.loadButton)
        controlLayout.addStretch()
        mainLayout.addLayout(controlLayout)

        # Data Table
        self.dataTable = QTableWidgetItem(0, 5)
        self.dataTable.setHorizontalHeaderLabels(["Source", "Destination",
"SNi", "JA3 Hash", "Matched Client"])

self.dataTable.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.S
tretch)
        mainLayout.addWidget(self.dataTable)

        def selectPcapFile(self):
            filePath, _ = QFileDialog.getOpenFileName(self, "Open PCAP",
"../pcaps", "PCAP Files (*.pcap *.pcapng)")
            if filePath:
                self.dataTable.setRowCount(0) # Clear previous results
                self.workerThread = PcapWorker(filePath)
                self.workerThread.rowExtracted.connect(self.appendTableRow)
                self.workerThread.start()

        def appendTableRow(self, packetData):
            currentRow = self.dataTable.rowCount()
            self.dataTable.insertRow(currentRow)
            self.dataTable.setItem(currentRow, 0,
QTableWidgetItem(packetData["source"]))
            self.dataTable.setItem(currentRow, 1,
QTableWidgetItem(packetData["destination"]))
            self.dataTable.setItem(currentRow, 2,
QTableWidgetItem(packetData["sni"]))
            self.dataTable.setItem(currentRow, 3,
QTableWidgetItem(packetData["ja3Hash"]))
            self.dataTable.setItem(currentRow, 4,
QTableWidgetItem(packetData["matchedClient"]))

if __name__ == "__main__":
    app = QApplication(sys.argv)
    monitorWindow = TlsMonitorGui()
    monitorWindow.show()
    sys.exit(app.exec())

```

**3. Run the App** Open your Windows terminal, ensure your virtual environment is active, navigate to code/python/, and execute: python gui.py

Click the load button and select cloudflare\_run1.pcap from the pcaps folder. You should instantly see a

structured grid populating with the extracted handshake identities.

Once this base is rendering correctly, we can easily add a details pane at the bottom to display the raw strings whenever you click on a specific row.

---

*User prompt: I want gui to be in a separate folder called gui. Dont try to go against me.*

Response: Set up a dedicated gui directory inside code/ (or at the repository root). To allow the GUI script inside code/gui/ to import the modules from code/python/src/, add the code/python directory to sys.path.

## Directory Structure

```
code/
├── gui/
│   └── main.py          <-- Your PyQt6 GUI application
├── python/
│   ├── fingerprints.json
│   └── src/
│       ├── capture.py
│       ├── db.py
│       └── ja3.py
└── pcaps/
    └── cloudflare_run1.pcap
```

## code/gui/main.py

Create main.py inside code/gui/:

```
import sys
from pathlib import Path
from PyQt6.QtWidgets import (
    QApplication,
    QMainWindow,
    QWidget,
    QVBoxLayout,
    QHBoxLayout,
    QPushButton,
    QTableWidgetItem,
    QTableWidgetItem,
    QFileDialog,
    QHeaderView,
)
from PyQt6.QtCore import QThread, pyqtSignal

# Resolve path to code/python so we can import the capture and ja3 modules
currentDir = Path(__file__).resolve().parent
pythonEngineDir = currentDir.parent / "python"
```

```

if str(pythonEngineDir) not in sys.path:
    sys.path.insert(0, str(pythonEngineDir))

from src.capture import read_pcap, CaptureStats
from src.ja3 import compute_ja3_hash
from src.db import FingerprintDB

class PcapWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    captureFinished = pyqtSignal(str)

    def __init__(self, pcapPath: str, dbPath: Path):
        super().__init__()
        self.pcapPath = pcapPath
        self.fingerprintDb = FingerprintDB(str(dbPath))
        self.captureStats = CaptureStats()

    def run(self):
        for result in read_pcap(self.pcapPath, self.captureStats):
            if result.client_hello:
                clientHello = result.client_hello
                ja3Hash = compute_ja3_hash(clientHello)
                match = self.fingerprintDb.lookup(ja3Hash) or "Unknown"

                packetData = {
                    "source": f"{result.src_ip}:{result.src_port}",
                    "destination": f"{result.dst_ip}:{result.dst_port}",
                    "sni": clientHello.server_name or "N/A",
                    "ja3Hash": ja3Hash,
                    "matchedClient": match,
                }
                self.rowExtracted.emit(packetData)

        self.captureFinished.emit(self.captureStats.summary())

class TlsMonitorGui(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(1000, 600)

        self.dbPath = pythonEngineDir / "fingerprints.json"
        self.workerThread = None

        mainWidget = QWidget()
        self.setCentralWidget(mainWidget)
        mainLayout = QVBoxLayout(mainWidget)

        # Top Controls
        controlLayout = QHBoxLayout()

```

```

self.loadButton = QPushButton("Load PCAP File")
self.loadButton.clicked.connect(self.selectPcapFile)
controlLayout.addWidget(self.loadButton)
controlLayout.addStretch()
mainLayout.addLayout(controlLayout)

# Data Table
self.dataTable = QTableWidgetItem(0, 5)
self.dataTable.setHorizontalHeaderLabels([
    "Source",
    "Destination",
    "SNI",
    "JA3 Hash",
    "Matched Client",
])
self.dataTable.horizontalHeader().setSectionResizeMode(
    QHeaderView.ResizeMode.Stretch
)
mainLayout.addWidget(self.dataTable)

def selectPcapFile(self):
    defaultPcapDir = str(currentDir.parent / "pcaps")
    filePath, _ = QFileDialog.getOpenFileName(
        self,
        "Open PCAP",
        defaultPcapDir,
        "PCAP Files (*.pcap *.pcapng)",
    )
    if filePath:
        self.dataTable.setRowCount(0)
        self.workerThread = PcapWorker(filePath, self.dbPath)
        self.workerThread.rowExtracted.connect(self.appendTableRow)
        self.workerThread.start()

    def appendTableRow(self, packetData):
        currentRow = self.dataTable.rowCount()
        self.dataTable.insertRow(currentRow)
        self.dataTable.setItem(currentRow, 0,
            QTableWidgetItem(packetData["source"]))
        self.dataTable.setItem(currentRow, 1,
            QTableWidgetItem(packetData["destination"]))
        self.dataTable.setItem(currentRow, 2,
            QTableWidgetItem(packetData["sni"]))
        self.dataTable.setItem(currentRow, 3,
            QTableWidgetItem(packetData["ja3Hash"]))
        self.dataTable.setItem(currentRow, 4,
            QTableWidgetItem(packetData["matchedClient"]))

if __name__ == "__main__":
    app = QApplication(sys.argv)
    monitorWindow = TlsMonitorGui()

```

```
monitorWindow.show()
sys.exit(app.exec())
```

## Running the App

In your Windows terminal (PowerShell or Command Prompt) with the virtual environment activated:

```
cd code\gui
python main.py
```

---

*User prompt: Boom, this is what it looks like. Here's what I want you to do. I want you to tell me how to get the whole thing working, so that I can see these records appear on the screen.*

Response: To test it immediately and populate the table:

1. Click the **Load PCAP File** button in the top left.
2. In the file dialog, navigate into code/pcaps/.
3. Select any capture file with handshakes—**curl.pcap**, **chrome.pcap**, or **cloudflare\_run1.pcap**.

The worker thread will parse the packets through `src.capture`, compute the JA3 hash, query `fingerprints.json`, and stream rows into your table.

## If nothing appears when selecting a file

The `QThread` execution swallows exceptions by default if an error occurs inside `run()`. Add a quick error hook and print statement to see any issues in your console:

In `code/gui/main.py`:

```
class PcapWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    captureFinished = pyqtSignal(str)
    errorOccurred = pyqtSignal(str)

    def __init__(self, pcapPath: str, dbPath: Path):
        super().__init__()
        self.pcapPath = pcapPath
        self.fingerprintDb = FingerprintDB(str(dbPath))
        self.captureStats = CaptureStats()

    def run(self):
        try:
            for result in read_pcap(self.pcapPath, self.captureStats):
                if result.client_hello:
                    clientHello = result.client_hello
                    ja3Hash = compute_ja3_hash(clientHello)
                    match = self.fingerprintDb.lookup(ja3Hash) or "Unknown"
```

```

        packetData = {
            "source": f"{result.src_ip}:{result.src_port}",
            "destination": f"{result.dst_ip}:{result.dst_port}",
            "sni": clientHello.server_name or "N/A",
            "ja3Hash": ja3Hash,
            "matchedClient": match,
        }
        self.rowExtracted.emit(packetData)

    self.captureFinished.emit(self.captureStats.summary())
except Exception as e:
    self.errorOccurred.emit(str(e))

```

Connect the error signal in selectPcapFile:

```

self.workerThread.errorOccurred.connect(lambda err: print(f"[!] Worker error: {err}"))
self.workerThread.captureFinished.connect(lambda summary: print(f"[+] Done:\n{summary}"))

```

## Adding Row Selection & Details Inspection

Once rows are displaying, you will want to view the raw cipher list, extensions, and the unhashed JA3 string.

Add a bottom inspection pane by updating your TlsMonitorGui.\_\_init\_\_ layout:

```

# Beneath self.dataTable in mainLayout:
self.detailsTextEdit = QTextEdit()
self.detailsTextEdit.setReadOnly(True)
self.detailsTextEdit.setFixedHeight(180)
self.detailsTextEdit.setPlaceholderText("Select any row above to inspect raw handshake metadata...")
mainLayout.addWidget(self.detailsTextEdit)

# Connect row clicks:
self.dataTable.itemSelectionChanged.connect(self.displaySelectedDetails)

```

Store the complete packet objects in a cache dictionary indexed by row (self.rowDataCache = {}) when appending, and format the details inside displaySelectedDetails.

Try loading curl.pcap first—do the records display in the table?

---

User prompt: *Hmm it only works with some pcap files, on others, it gives this (.venv) PS E:\CS331-T014-TLS\_Fingerprinting\code\gui> python main.py Traceback (most recent call last): File*

```
import os

def read_pcap(
    filepath: str,
    stats: CaptureStats | None = None,
) -> Iterator[TLShandshakeResult]:
```

```

if stats is None:
    stats = CaptureStats()

# Guard against 0-byte or truncated files
if not os.path.exists(filepath) or os.path.getsize(filepath) < 24:
    logger.warning(f"File {filepath} is empty or smaller than a 24-byte
PCAP header.")
    return

reassembler = TCPReassembler(stats)
# ... rest of your read_pcap function remains unchanged

```

## How to Implement Live Packet Capture ("On the Go")

Live capture works by feeding packets from a live network interface into your existing TCPReassembler in real time, rather than reading them from a file.

### 1. Windows Prerequisite: Npcap

Windows does not support raw promiscuous packet sniffing through standard OS sockets.

- Download and install **Npcap** (npcap.com).
- During installation, check the box: **"Install Npcap in WinPcap API-compatible Mode"**.

### 2. Live Capture Worker (Scapy + TCPReassembler)

scapy is already in requirements.txt. Scapy provides a streaming sniffer (AsyncSniffer) that catches raw frames on your active network adapter and extracts the IP and TCP headers.

Here is how live packets are adapted to TCPReassembler.process\_packet():

```

from scapy.all import AsyncSniffer, IP, IPv6, TCP
from PyQt6.QtCore import QThread, pyqtSignal
from src.capture import TCPReassembler, CaptureStats
from src.ja3 import compute_ja3_hash, compute_ja3_string
from src.db import FingerprintDB

class LiveCaptureWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    statusUpdated = pyqtSignal(str)

    def __init__(self, interfaceName: str | None, dbPath: str):
        super().__init__()
        self.interfaceName = interfaceName
        self.fingerprintDb = FingerprintDB(dbPath)
        self.captureStats = CaptureStats()
        self.reassembler = TCPReassembler(self.captureStats)
        self.sniffer = None
        self.isRunning = True

```

```

def processLivePacket(self, pkt):
    if not (pkt.haslayer(TCP) and (pkt.haslayer(IP) or
pkt.haslayer(IPv6))):
        return

    ipLayer = pkt[IP] if pkt.haslayer(IP) else pkt[IPv6]
    tcpLayer = pkt[TCP]

    srcIp = str(ipLayer.src)
    dstIp = str(ipLayer.dst)
    srcPort = int(tcpLayer.sport)
    dstPort = int(tcpLayer.dport)
    seq = int(tcpLayer.seq)
    flags = int(tcpLayer.flags)
    payload = bytes(tcpLayer.payload)
    timestamp = float(pkt.time)

    results = self.reassembler.process_packet(
        srcIp, dstIp, srcPort, dstPort, seq, flags, payload, timestamp
    )

    for res in results:
        if res.client_hello:
            ch = res.client_hello
            ja3Hash = compute_ja3_hash(ch)
            match = self.fingerprintDb.lookup(ja3Hash) or "Unknown"

            packetData = {
                "source": f"{res.src_ip}:{res.src_port}",
                "destination": f"{res.dst_ip}:{res.dst_port}",
                "sni": ch.server_name or "N/A",
                "ja3Hash": ja3Hash,
                "matchedClient": match,
                "ja3Raw": compute_ja3_string(ch),
            }
            self.rowExtracted.emit(packetData)

def run(self):
    self.statusUpdated.emit("Sniffing active (filtering tcp port 443)...")
    # BPF filter ensures we only sniff TLS traffic to conserve CPU
    self.sniffer = AsyncSniffer(
        iface=self.interfaceName,
        filter="tcp port 443",
        prn=self.processLivePacket,
        store=False
    )
    self.sniffer.start()

    while self.isRunning:
        self.msleep(100)

```

```

        if self.sniffer.running:
            self.sniffer.stop()

def stopCapture(self):
    self.isRunning = False
    self.wait()

```

## Integrating Live Sniffing into code/gui/main.py

Update code/gui/main.py to add interface selection and Start/Stop controls alongside the PCAP loader:

```

import sys
import os
from pathlib import Path
from PyQt6.QtWidgets import (
    QApplication,
    QMainWindow,
    QWidget,
    QVBoxLayout,
    QHBoxLayout,
    QPushButton,
    QTableWidgetItem,
    QTableWidgetItemItem,
    QFileDialog,
    QHeaderView,
    QComboBox,
    QLabel,
    QTextEdit
)
from PyQt6.QtCore import QThread, pyqtSignal
from scapy.all import get_if_list

currentDir = Path(__file__).resolve().parent
pythonEngineDir = currentDir.parent / "python"
if str(pythonEngineDir) not in sys.path:
    sys.path.insert(0, str(pythonEngineDir))

from src.capture import read_pcap, TCPReassembler, CaptureStats
from src.ja3 import compute_ja3_hash, compute_ja3_string
from src.db import FingerprintDB

class PcapWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    captureFinished = pyqtSignal(str)

    def __init__(self, pcapPath: str, dbPath: Path):
        super().__init__()
        self.pcapPath = pcapPath
        self.fingerprintDb = FingerprintDB(str(dbPath))

```

```

        self.captureStats = CaptureStats()

    def run(self):
        if not os.path.exists(self.pcapPath) or os.path.getsize(self.pcapPath)
< 24:
            self.captureFinished.emit("Selected file is empty or corrupted.")
            return

        for result in read_pcap(self.pcapPath, self.captureStats):
            if result.client_hello:
                clientHello = result.client_hello
                ja3Hash = compute_ja3_hash(clientHello)
                match = self.fingerprintDb.lookup(ja3Hash) or "Unknown"

                packetData = {
                    "source": f"{result.src_ip}:{result.src_port}",
                    "destination": f"{result.dst_ip}:{result.dst_port}",
                    "sni": clientHello.server_name or "N/A",
                    "ja3Hash": ja3Hash,
                    "matchedClient": match,
                    "ja3Raw": compute_ja3_string(clientHello),
                }
                self.rowExtracted.emit(packetData)

        self.captureFinished.emit(self.captureStats.summary())

class LiveCaptureWorker(QThread):
    rowExtracted = pyqtSignal(dict)

    def __init__(self, interfaceName: str, dbPath: Path):
        super().__init__()
        self.interfaceName = interfaceName
        self.fingerprintDb = FingerprintDB(str(dbPath))
        self.captureStats = CaptureStats()
        self.reassembler = TCPReassembler(self.captureStats)
        self.sniffer = None
        self.isRunning = True

    def processLivePacket(self, pkt):
        from scapy.all import IP, IPv6, TCP
        if not (pkt.haslayer(TCP) and (pkt.haslayer(IP) or
pkt.haslayer(IPv6))):
            return

        ipLayer = pkt[IP] if pkt.haslayer(IP) else pkt[IPv6]
        tcpLayer = pkt[TCP]

        srcIp = str(ipLayer.src)
        dstIp = str(ipLayer.dst)
        srcPort = int(tcpLayer.sport)
        dstPort = int(tcpLayer.dport)

```

```

seq = int(tcpLayer.seq)
flags = int(tcpLayer.flags)
payload = bytes(tcpLayer.payload)
timestamp = float(pkt.time)

results = self.reassembler.process_packet(
    srcIp, dstIp, srcPort, dstPort, seq, flags, payload, timestamp
)

for res in results:
    if res.client_hello:
        ch = res.client_hello
        ja3Hash = compute_ja3_hash(ch)
        match = self.fingerprintDb.lookup(ja3Hash) or "Unknown"

        packetData = {
            "source": f"{res.src_ip}:{res.src_port}",
            "destination": f"{res.dst_ip}:{res.dst_port}",
            "sni": ch.server_name or "N/A",
            "ja3Hash": ja3Hash,
            "matchedClient": match,
            "ja3Raw": compute_ja3_string(ch),
        }
        self.rowExtracted.emit(packetData)

def run(self):
    from scapy.all import AsyncSniffer
    self.sniffer = AsyncSniffer(
        iface=self.interfaceName,
        filter="tcp port 443",
        prn=self.processLivePacket,
        store=False
    )
    self.sniffer.start()
    while self.isRunning:
        self.msleep(100)

    if self.sniffer and self.sniffer.running:
        self.sniffer.stop()

def stopCapture(self):
    self.isRunning = False
    self.wait()

class TlsMonitorGui(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(1100, 650)

        self.dbPath = pythonEngineDir / "fingerprints.json"

```

```

self.pcapWorker = None
self.liveWorker = None
self.rowCache = []

mainWidget = QWidget()
self.setCentralWidget(mainWidget)
mainLayout = QVBoxLayout(mainWidget)

controlLayout = QHBoxLayout()

# Offline PCAP button
self.loadButton = QPushButton("Load PCAP File")
self.loadButton.clicked.connect(self.selectPcapFile)
controlLayout.addWidget(self.loadButton)

# Live interface selection
controlLayout.addWidget(QLabel("Interface:"))
self.interfaceDropdown = QComboBox()
try:
    self.interfaceDropdown.addItem(get_if_list())
except Exception:
    self.interfaceDropdown.addItem("Default")
controlLayout.addWidget(self.interfaceDropdown)

self.liveButton = QPushButton("Start Live Capture")
self.liveButton.clicked.connect(self.toggleLiveCapture)
controlLayout.addWidget(self.liveButton)

controlLayout.addStretch()
mainLayout.addLayout(controlLayout)

# Handshake Table
self.dataTable = QTableWidgetItem(0, 5)
self.dataTable.setHorizontalHeaderLabels([
    "Source",
    "Destination",
    "SNI",
    "JA3 Hash",
    "Matched Client",
])
self.dataTable.horizontalHeader().setSectionResizeMode(
    QHeaderView.ResizeMode.Stretch
)
self.dataTable.itemSelectionChanged.connect(self.displayRowDetails)
mainLayout.addWidget(self.dataTable)

# Raw Inspection Pane
self.detailsPane = QTextEdit()
self.detailsPane.setReadOnly(True)
self.detailsPane.setFixedHeight(120)
self.detailsPane.setPlaceholderText("Select any handshake above to view
raw metadata and JA3 string.")

```

```

        mainLayout.addWidget(self.detailsPane)

def selectPcapFile(self):
    defaultPcapDir = str(currentDir.parent / "pcaps")
    filePath, _ = QFileDialog.getOpenFileName(
        self,
        "Open PCAP",
        defaultPcapDir,
        "PCAP Files (*.pcap *.pcapng)",
    )
    if filePath:
        self.dataTable.setRowCount(0)
        self.rowCache.clear()
        self.detailsPane.clear()

        self.pcapWorker = PcapWorker(filePath, self.dbPath)
        self.pcapWorker.rowExtracted.connect(self.appendTableRow)
        self.pcapWorker.start()

def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning:
        self.liveWorker.stopCapture()
        self.liveWorker = None
        self.liveButton.setText("Start Live Capture")
        self.loadButton.setEnabled(True)
    else:
        selectedInterface = self.interfaceDropdown.currentText()
        self.dataTable.setRowCount(0)
        self.rowCache.clear()
        self.detailsPane.clear()

        self.liveWorker = LiveCaptureWorker(selectedInterface, self.dbPath)
        self.liveWorker.rowExtracted.connect(self.appendTableRow)
        self.liveWorker.start()

        self.liveButton.setText("Stop Live Capture")
        self.loadButton.setEnabled(False)

def appendTableRow(self, packetData):
    currentRow = self.dataTable.rowCount()
    self.dataTable.insertRow(currentRow)
    self.rowCache.append(packetData)

    self.dataTable.setItem(currentRow, 0,
        QTableWidgetItem(packetData["source"]))
    self.dataTable.setItem(currentRow, 1,
        QTableWidgetItem(packetData["destination"]))
    self.dataTable.setItem(currentRow, 2,
        QTableWidgetItem(packetData["sni"]))
    self.dataTable.setItem(currentRow, 3,
        QTableWidgetItem(packetData["ja3Hash"]))
    self.dataTable.setItem(currentRow, 4,

```

```

QTableWidgetItem(packetData["matchedClient"]))

def displayRowDetails(self):
    selectedRows = self.dataTable.selectionModel().selectedRows()
    if not selectedRows:
        return
    rowIndex = selectedRows[0].row()
    if rowIndex < len(self.rowCache):
        entry = self.rowCache[rowIndex]
        detailText = (
            f"SNI:           {entry['sni']}\n"
            f"JA3 Hash:         {entry['ja3Hash']}\n"
            f"Client:            {entry['matchedClient']}\n"
            f"Raw JA3:          {entry.get('ja3Raw', 'N/A')}"
        )
        self.detailsPane.setText(detailText)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    monitorWindow = TlsMonitorGui()
    monitorWindow.show()
    sys.exit(app.exec())

```

## Running the Live Pipeline

1. Install Npcap in WinPcap-compatible mode.
2. Run `python main.py` in `code/gui`.
3. Select your active Wi-Fi or Ethernet adapter in the dropdown and click **Start Live Capture**.
4. Open a browser or run `curl -I [https://cloudflare.com](https://cloudflare.com)` in a separate terminal. The extracted TLS ClientHello rows will appear live in the table.

*User prompt: You gave me all this code fr "Live Capture Worker". What do you want me to do with that? Also, remember that I like the code to be modular and split across files meant for specific things. I have just installed npcap for windows*

Response: Since you prefer a modular architecture and have Npcap installed, we will split the monolithic script into three distinct files inside your `code/gui/` directory. Npcap will automatically bind to Scapy to enable the live capture worker.

**Directory Structure** Create the following files inside `code/gui/`:

- `main.py` (The application entry point)
- `window.py` (The UI layout and event routing)
- `workers.py` (The background QThread processes)

**code/gui/workers.py** This handles all background parsing so the UI never freezes. It imports your existing Python engine and uses camelCase for all new variables and methods.

```

import os
from PyQt6.QtCore import QThread, pyqtSignal
from scapy.all import AsyncSniffer, IP, IPv6, TCP

from src.capture import read_pcap, TCPReassembler, CaptureStats
from src.ja3 import compute_ja3_hash, compute_ja3_string
from src.db import FingerprintDB

class PcapWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    captureFinished = pyqtSignal(str)
    errorOccurred = pyqtSignal(str)

    def __init__(self, pcapPath, dbPath):
        super().__init__()
        self.pcapPath = pcapPath
        self.fingerprintDb = FingerprintDB(str(dbPath))
        self.captureStats = CaptureStats()

    def run(self):
        try:
            if not os.path.exists(self.pcapPath) or
os.path.getsize(self.pcapPath) < 24:
                self.errorOccurred.emit("Selected file is empty or corrupted.")
                return

            for result in read_pcap(self.pcapPath, self.captureStats):
                if result.client_hello:
                    clientHello = result.client_hello
                    ja3Hash = compute_ja3_hash(clientHello)
                    match = self.fingerprintDb.lookup(ja3Hash) or "Unknown"

                    packetData = {
                        "source": f"{result.src_ip}:{result.src_port}",
                        "destination": f"{result.dst_ip}:{result.dst_port}",
                        "sni": clientHello.server_name or "N/A",
                        "ja3Hash": ja3Hash,
                        "matchedClient": match,
                        "ja3Raw": compute_ja3_string(clientHello),
                    }
                    self.rowExtracted.emit(packetData)

                self.captureFinished.emit(self.captureStats.summary())
        except Exception as e:
            self.errorOccurred.emit(str(e))

class LiveCaptureWorker(QThread):
    rowExtracted = pyqtSignal(dict)

    def __init__(self, interfaceName, dbPath):
        super().__init__()
        self.interfaceName = interfaceName

```

```

self.fingerprintDb = FingerprintDB(str(dbPath))
self.captureStats = CaptureStats()
self.reassembler = TCPReassembler(self.captureStats)
self.sniffer = None
self.isRunning = True

def processLivePacket(self, pkt):
    if not (pkt.haslayer(TCP) and (pkt.haslayer(IP) or
pkt.haslayer(IPv6))):
        return

    ipLayer = pkt[IP] if pkt.haslayer(IP) else pkt[IPv6]
    tcpLayer = pkt[TCP]

    srcIp = str(ipLayer.src)
    dstIp = str(ipLayer.dst)
    srcPort = int(tcpLayer.sport)
    dstPort = int(tcpLayer.dport)
    seq = int(tcpLayer.seq)
    flags = int(tcpLayer.flags)
    payload = bytes(tcpLayer.payload)
    timestamp = float(pkt.time)

    results = self.reassembler.process_packet(
        srcIp, dstIp, srcPort, dstPort, seq, flags, payload, timestamp
    )

    for res in results:
        if res.client_hello:
            clientHello = res.client_hello
            ja3Hash = compute_ja3_hash(clientHello)
            match = self.fingerprintDb.lookup(ja3Hash) or "Unknown"

            packetData = {
                "source": f"{res.src_ip}:{res.src_port}",
                "destination": f"{res.dst_ip}:{res.dst_port}",
                "sni": clientHello.server_name or "N/A",
                "ja3Hash": ja3Hash,
                "matchedClient": match,
                "ja3Raw": compute_ja3_string(clientHello),
            }
            self.rowExtracted.emit(packetData)

def run(self):
    self.sniffer = AsyncSniffer(
        iface=self.interfaceName,
        filter="tcp port 443",
        prn=self.processLivePacket,
        store=False
    )
    self.sniffer.start()
    while self.isRunning:

```

```

        self.msleep(100)

    if self.sniffer and self.sniffer.running:
        self.sniffer.stop()

    def stopCapture(self):
        self.isRunning = False
        self.wait()

```

**code/gui/window.py** This controls the layout, dropdowns, and table logic.

```

from pathlib import Path
from PyQt6.QtWidgets import (
    QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
    QTableWidgetItem, QTableWidgetItem, QFileDialog, QHeaderView,
    QComboBox, QLabel, QTextEdit
)
from scapy.all import get_if_list
from workers import PcapWorker, LiveCaptureWorker

class TlsMonitorGui(QMainWindow):
    def __init__(self, pythonEngineDir):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(1100, 650)

        self.dbPath = pythonEngineDir / "fingerprints.json"
        self.pcapsDir = pythonEngineDir.parent / "pcaps"
        self.pcapWorker = None
        self.liveWorker = None
        self.rowCache = []

        mainWidget = QWidget()
        self.setCentralWidget(mainWidget)
        mainLayout = QVBoxLayout(mainWidget)

        controlLayout = QHBoxLayout()

        self.loadButton = QPushButton("Load PCAP File")
        self.loadButton.clicked.connect(self.selectPcapFile)
        controlLayout.addWidget(self.loadButton)

        controlLayout.addWidget(QLabel("Interface:"))
        self.interfaceDropdown = QComboBox()
        try:
            self.interfaceDropdown.addItem(get_if_list())
        except Exception:
            self.interfaceDropdown.addItem("Default")
        controlLayout.addWidget(self.interfaceDropdown)

        self.liveButton = QPushButton("Start Live Capture")

```

```

self.liveButton.clicked.connect(self.toggleLiveCapture)
controlLayout.addWidget(self.liveButton)

controlLayout.addStretch()
mainLayout.addLayout(controlLayout)

self.dataTable = QTableWidgetItem(0, 5)
self.dataTable.setHorizontalHeaderLabels([
    "Source", "Destination", "SNI", "JA3 Hash", "Matched Client"
])

self.dataTable.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.S
tretch)
self.dataTable.itemSelectionChanged.connect(self.displayRowDetails)
mainLayout.addWidget(self.dataTable)

self.detailsPane = QTextEdit()
self.detailsPane.setReadOnly(True)
self.detailsPane.setFixedHeight(120)
self.detailsPane.setPlaceholderText("Select any handshake above to view
raw metadata and JA3 string.")
mainLayout.addWidget(self.detailsPane)

def selectPcapFile(self):
    filePath, _ = QFileDialog.getOpenFileName(
        self, "Open PCAP", str(self.pcapsDir), "PCAP Files (*.pcap
*.pcapng)"
    )
    if filePath:
        self.dataTable.setRowCount(0)
        self.rowCache.clear()
        self.detailsPane.clear()

        self.pcapWorker = PcapWorker(filePath, self.dbPath)
        self.pcapWorker.rowExtracted.connect(self.appendTableRow)
        self.pcapWorker.errorOccurred.connect(lambda err:
self.detailsPane.setText(f"Error: {err}"))
        self.pcapWorker.start()

def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning:
        self.liveWorker.stopCapture()
        self.liveWorker = None
        self.liveButton.setText("Start Live Capture")
        self.loadButton.setEnabled(True)
    else:
        selectedInterface = self.interfaceDropdown.currentText()
        self.dataTable.setRowCount(0)
        self.rowCache.clear()
        self.detailsPane.clear()

        self.liveWorker = LiveCaptureWorker(selectedInterface, self.dbPath)

```

```

        self.liveWorker.rowExtracted.connect(self.appendTableRow)
        self.liveWorker.start()

        self.liveButton.setText("Stop Live Capture")
        self.loadButton.setEnabled(False)

    def appendTableRow(self, packetData):
        currentRow = self.dataTable.rowCount()
        self.dataTable.insertRow(currentRow)
        self.rowCache.append(packetData)

        self.dataTable.setItem(currentRow, 0,
                                QTableWidgetItem(packetData["source"]))
        self.dataTable.setItem(currentRow, 1,
                                QTableWidgetItem(packetData["destination"]))
        self.dataTable.setItem(currentRow, 2,
                                QTableWidgetItem(packetData["sni"]))
        self.dataTable.setItem(currentRow, 3,
                                QTableWidgetItem(packetData["ja3Hash"]))
        self.dataTable.setItem(currentRow, 4,
                                QTableWidgetItem(packetData["matchedClient"]))

    def displayRowDetails(self):
        selectedRows = self.dataTable.selectionModel().selectedRows()
        if not selectedRows:
            return
        rowIndex = selectedRows[0].row()
        if rowIndex < len(self.rowCache):
            entry = self.rowCache[rowIndex]
            detailText = (
                f"SNI:          {entry['sni']}\n"
                f"JA3 Hash:       {entry['ja3Hash']}\n"
                f"Client:         {entry['matchedClient']}\n"
                f"Raw JA3:        {entry.get('ja3Raw', 'N/A')}"
            )
            self.detailsPane.setText(detailText)

```

**code/gui/main.py** This acts purely as the launcher, resolving your import paths so the GUI can "see" your Python backend engine.

```

import sys
from pathlib import Path
from PyQt6.QtWidgets import QApplication

# Resolve paths before importing window
currentDir = Path(__file__).resolve().parent
pythonEngineDir = currentDir.parent / "python"
if str(pythonEngineDir) not in sys.path:
    sys.path.insert(0, str(pythonEngineDir))

from window import TlsMonitorGui

```

```

if __name__ == "__main__":
    app = QApplication(sys.argv)
    monitorWindow = TlsMonitorGui(pythonEngineDir)
    monitorWindow.show()
    sys.exit(app.exec())

```

---

*User prompt: Why am I not getting any records coming up here*

Response: Live packet capture only intercepts the very first moment of a connection (the ClientHello handshake). If you are simply watching the screen without actively browsing, no new handshakes are being generated.

Here is how to get the traffic flowing and troubleshoot common Windows capture issues:

- **Force a New Handshake:** Open a completely new incognito/private browser window and navigate to a site you haven't visited recently (e.g., [https://cloudflare.com](https://cloudflare.com)). Existing open tabs keep their connections alive (keep-alive) and will not generate the new handshakes your tool is looking for.
- **Select the Correct Adapter:** Windows creates dozens of virtual network adapters for WSL, Docker, and Bluetooth. Stop the capture, click the **Interface** dropdown, and try the next \Device\NPF\_{...} in the list until you find your actual active Wi-Fi or Ethernet adapter.
- **Run as Administrator:** Npcap often silently drops packets if the Python script does not have elevated privileges. Close your terminal, right-click it, select **Run as Administrator**, and launch python main.py again.
- **Surface Silent Scapy Errors:** If Scapy crashes while trying to bind to an invalid Windows interface, the QThread swallows the error silently. You can catch this by adding a try/except block to your LiveCaptureWorker's run() method in workers.py:

```

def run(self):
    from scapy.all import AsyncSniffer
    try:
        self.sniffer = AsyncSniffer(
            iface=self.interfaceName,
            filter="tcp port 443",
            prn=self.processLivePacket,
            store=False
        )
        self.sniffer.start()
        while self.isRunning:
            self.msleep(100)

        if self.sniffer and self.sniffer.running:
            self.sniffer.stop()
    except Exception as e:
        # You will need to add errorOccurred = pyqtSignal(str) to the class
        # and connect it to your detailsPane in window.py, just like
PcapWorker
        self.errorOccurred.emit(f"Scapy Sniffer Error: {str(e)}")

```

---

*User prompt: idk I havent done what you said other than running as administrator. These are the listed interfaces. I have also opened up different sites looking for something. Thing is when I open up wireshark, I can see thousands of packages coming in. Can you please tell me how I can fix this*

Response: The reason you aren't seeing any records is almost certainly because you are caught in the "Windows GUID Guessing Game."

Windows doesn't pass friendly interface names (like "Wi-Fi" or "Ethernet") to packet sniffers by default; it passes those long `\Device\NPF_{GUID}` strings. You are likely sniffing a dormant virtual adapter (like a disconnected Bluetooth or WSL bridge) rather than your active internet connection.

Here is how to fix this immediately, and how to update your code so you never have to guess again.

**1. The Quick Fix: Use Wireshark as a Decoder Ring** Since you already have Wireshark open, use it to find the exact GUID of your active connection:

- In Wireshark, go to **Capture > Options** (or click the gear icon).
- Find the interface that shows active traffic (e.g., "Wi-Fi").
- Hover over it or look at the "Interface Name" column. It will show you the exact `\Device\NPF_{...}` string it uses.
- Select that exact string in your PyQt6 application dropdown.
- *Note:* Start the capture in your app **before** you navigate to a new site.

**2. The Permanent Fix: Update the GUI to Show Friendly Names** Instead of displaying raw GUIDs, we can use Scapy's Windows-specific registry parser to map them to human-readable names.

In `code/gui/window.py`, update the interface loading logic. Replace the current `self.interfaceDropdown` setup in the `__init__` method with this:

```
# Replace the get_if_list() block with this:
from scapy.arch.windows import get_windows_if_list

controlLayout.addWidget(QLabel("Interface:"))
self.interfaceDropdown = QComboBox()
self.interfaceMap = {} # Dictionary to map friendly names back to NPF
GUIDs

try:
    for iface in get_windows_if_list():
        friendlyName = iface.get('name', 'Unknown')
        guid = iface.get('netid', '')
        if guid:
            # Format: "Wi-Fi ({...GUID...})"
            displayName = f"{friendlyName} ({guid})"
            # Scapy needs the exact NPF string for sniffing
            self.interfaceMap[displayName] = rf"\Device\NPF_{guid}"
            self.interfaceDropdown.addItem(displayName)
except Exception as e:
```

```
self.interfaceDropdown.addItem("Default")

controlLayout.addWidget(self.interfaceDropdown)
```

Then, update your toggleLiveCapture method so it passes the raw GUID to the worker, not the friendly name:

```
def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning:
        # ... (keep existing stop logic)
    else:
        selectedDisplay = self.interfaceDropdown.currentText()
        # Retrieve the raw \Device\NPF_ string from the map
        selectedInterface = self.interfaceMap.get(selectedDisplay,
selectedDisplay)

        self.dataTable.setRowCount(0)
        self.rowCache.clear()
        self.detailsPane.clear()

        self.liveWorker = LiveCaptureWorker(selectedInterface, self.dbPath)
        self.liveWorker.rowExtracted.connect(self.appendTableRow)
        self.liveWorker.start()

        # ... (keep existing button toggle logic)
```

**3. The HTTP/3 (QUIC) Trap** If you have the correct interface selected and *still* see nothing, your browser might be upgrading connections to HTTP/3 (QUIC). QUIC uses **UDP** port 443, not TCP. Because your LiveCaptureWorker specifically filters for tcp port 443, those handshakes are entirely ignored.

To test if your engine is working perfectly, open your terminal and run `curl [https://example.com](https://example.com)` (since curl defaults to TCP TLS 1.2/1.3) or open an older website in a fresh incognito window.

---

*User prompt: ehhhh*

Response: To find the exact `\Device\NPF_{...}` string for your active "Wi-Fi" adapter using your open Wireshark window, follow these steps.

### Method 1: The Wireshark Lookup (Immediate Fix)

1. In Wireshark, click the **Gear icon** (Capture Options) located on the top left of the main toolbar.
2. In the "Input" tab that opens, find **Wi-Fi** in the list.
3. Look at the column labeled **Device Name** or **Interface Name** right next to it (you may need to drag the column header to expand it).
4. You will see the exact `\Device\NPF_{GUID}` string that corresponds to your Wi-Fi card.

5. Select that matching string in your Python application's dropdown.

**Method 2: Update the Code (Permanent Fix)** Because you mentioned skipping the code update in the previous step, your app is still pulling the raw, unreadable machine names. To make your dropdown display "Wi-Fi" just like Wireshark does, update the interface loading logic in code/gui/window.py inside your `__init__` function:

```
this:
    # Delete the old get_if_list() try/except block and replace it with
    from scapy.arch.windows import get_windows_if_list

    controlLayout.addWidget(QLabel("Interface:"))
    self.interfaceDropdown = QComboBox()
    self.interfaceMap = {}

    try:
        for iface in get_windows_if_list():
            friendlyName = iface.get('name', 'Unknown')
            guid = iface.get('netid', '')
            if guid:
                displayName = f"{friendlyName} ({guid})"
                self.interfaceMap[displayName] = rf"\Device\NPF_{guid}"
                self.interfaceDropdown.addItem(displayName)
    except Exception:
        self.interfaceDropdown.addItem("Default")

    controlLayout.addWidget(self.interfaceDropdown)
```

Then, update `toggleLiveCapture` in the same file to use the mapped dictionary:

```
def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning:
        self.liveWorker.stopCapture()
        self.liveWorker = None
        self.liveButton.setText("Start Live Capture")
        self.loadButton.setEnabled(True)
    else:
        selectedDisplay = self.interfaceDropdown.currentText()
        # This line translates "Wi-Fi" back to the \Device\NPF_ string
        Scapy needs
        selectedInterface = self.interfaceMap.get(selectedDisplay,
        selectedDisplay)

        self.dataTable.setRowCount(0)
        self.rowCache.clear()
        self.detailsPane.clear()

        self.liveWorker = LiveCaptureWorker(selectedInterface, self.dbPath)
        self.liveWorker.rowExtracted.connect(self.appendTableRow)
        self.liveWorker.start()
```

```
self.liveButton.setText("Stop Live Capture")  
self.loadButton.setEnabled(False)
```

---